

11/10/22

30 AGO 2021. 1

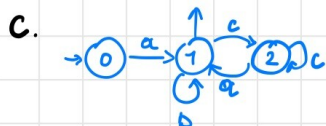
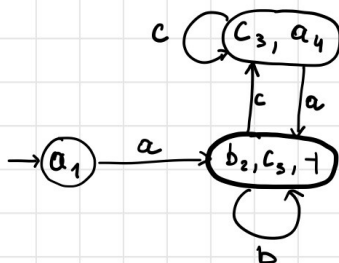
$$R = a(b|c^+a)^*$$

a. a
ab
acab
acab
acab
acab
acab
acab
acab
acab

$$b. R_{\#} = a_1(b_2|c_3^+a_4)^*+$$

$$I_{n1} = \{a_1\}$$

Term	Fol
a_1	$b_2, c_3, \neg$
b_2	$b_2, c_3, \neg$
c_3	c_3, a_4
a_4	$b_2, c_3, \neg$



	0	1	2
0	\		
1	\	\	
2	\	\	\

	1	2	
0	X	X	0
1	X	X	1

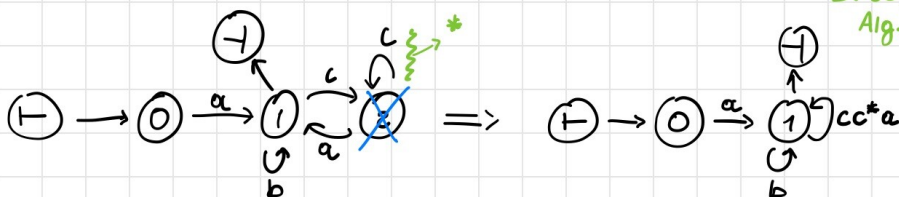
final vs. non final is by definition final

Alternative:

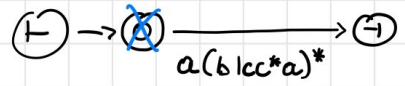
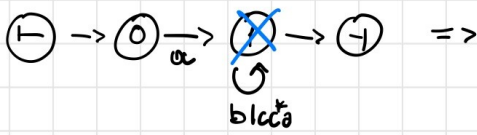
	0	1	2
a	1	-	1
b	-	1	-
c	-	2	2
!?	F	T	F

We look for identical columns, the states with equal columns are undistinguishable

d.



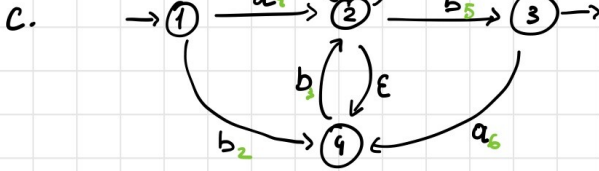
Brazerowski, Alg.



$$\Rightarrow R' = a(b|cc*a)^*$$

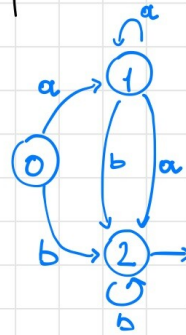
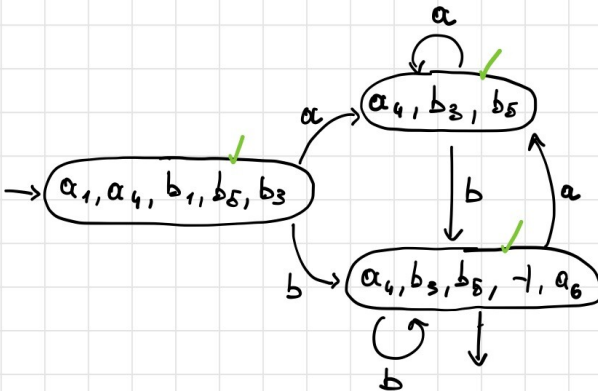
e. TODO

5 LUG 2017. 1



$$|n_1| = \{a_1, a_4, b_2, b_5, b_3\}$$

T	Fol
a_4, b_3, a_1	a_4, b_3, b_5
a_6, b_2	b_3
b_5	$\neg, a_6$



Removing ϵ -transitions:

1. BACKWARD PROPAGATION:



2. FORWARD PROPAGATION:



13/10/22

4 SEP 2015. 2.1

$$L = \{ xcwcy \mid x, y \in \{a, b\}^*, w^R = yx \}$$

$w = x^R y^R$

x	y	w ^R	w
a	ba	baa	aab
bbab	a	abba	baaba

a. $\begin{array}{c} \downarrow \\ \text{xcx}^R \text{y}^R \text{cy} \\ \downarrow \quad \downarrow \\ \text{PALINDROME} \quad \text{PALINDROME} \end{array}$

$$G = \begin{cases} S \rightarrow AA \\ A \rightarrow aAa \mid bAb \mid c \end{cases}$$

b. $L' = L \cap (ab)^* c \Sigma^* c (ab)^* \Rightarrow$

$\rightarrow \{a, b, c\}$

$$G' = \begin{cases} S \rightarrow AB \\ A \rightarrow abAba \mid c \\ B \rightarrow baBab \mid c \end{cases}$$

c. $L'' = \overline{L'}$

Context free languages are not closed w.r.t. complement.
 L'' must be context free because we can build a PDA that checks if we do not have the palindrome. This means that the language is not regular.

4 SEP 2015 2.2

1. $\langle \text{forest} \rangle \rightarrow (\langle \text{tree} \rangle ', ')^* \langle \text{tree} \rangle \leftarrow \text{quoting as, as a spec char} \quad \text{CONVENTION: AXIOM IS THE FIRST RULE}$

$\langle \text{tree} \rangle \rightarrow \langle \text{node1} \rangle$

$\langle \text{node1} \rangle \rightarrow \langle 3 \text{ node} \rangle \mid \langle \text{node2} \rangle$

$\langle \text{node2} \rangle \rightarrow \langle 2 \text{ node} \rangle \mid \langle \text{leaf} \rangle$

$\langle 3 \text{ node} \rangle \rightarrow ' \langle 3 \text{ TR START} \rangle ' \langle \text{node1} \rangle^3 ' \langle 3 \text{ TR END} \rangle '$

$\langle 2 \text{ node} \rangle \rightarrow ' \langle 2 \text{ TR START} \rangle ' \langle \text{node2} \rangle^2 ' \langle 2 \text{ TR END} \rangle '$

$\langle \text{leaf} \rangle \rightarrow \text{txt} \mid \text{nbr}$

AST

< 3 TRSTRT >

 $t \times t$

$\langle 2TRSTART \rangle$

76r

txc

<2 TREND>

 η br

< 2 TREND >

< forest >

< tree >

<node 1>

```
<?node:
```

'<3 TRSTRT>' <node1> <node1> <node1> '<8 TREND>'

<node2>

< heap >

$$t \times t$$

< node 2 >

(2 node)

'<2TRSTART' (node 2) (node 2) '<2TREND'

< heat >

nr

$\langle \text{leaf} \rangle$

txc

2. $\{n \text{ node}\} \rightarrow \{ \text{'L' STRSTRT} \} \{ \text{node 1} \}^3 \{ \text{node} \}^+ \{ \text{'L' STREND} \}$

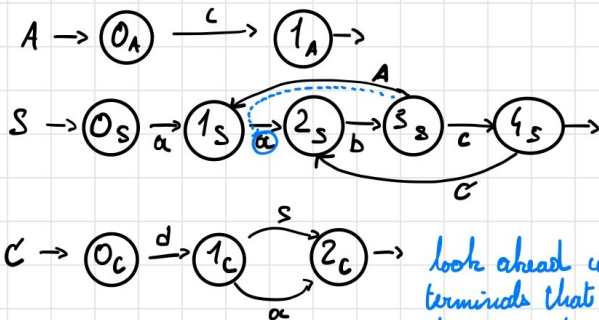
$\langle \text{node } 1 \rangle \rightarrow \langle \text{node } 2 \rangle \rightarrow \dots \rightarrow \langle \text{node } n \rangle$

10/11/22

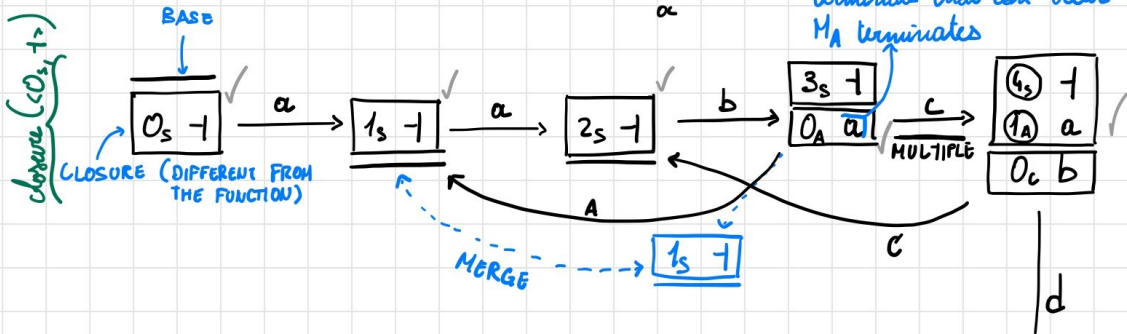
3 Mar - 3

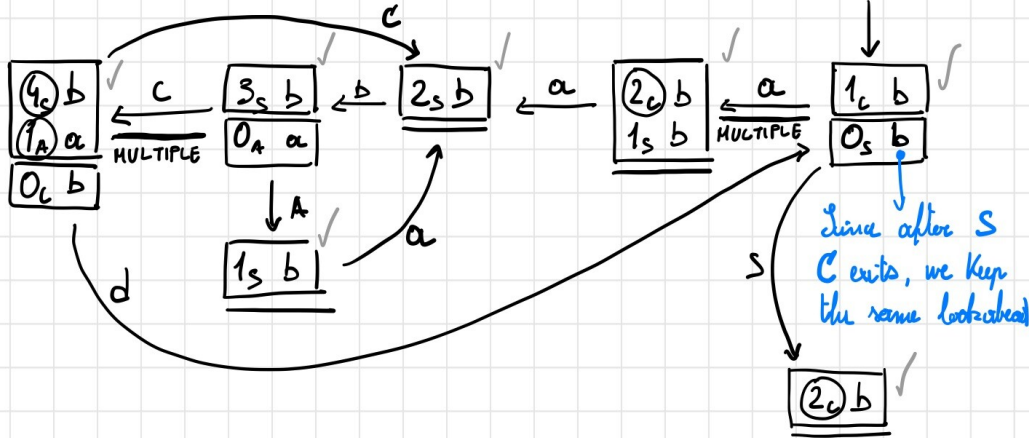
$$\Sigma = \{a, b, c, d\}$$

Grammar symbols:

$$\{a, b, c, d, A, S, C\}$$


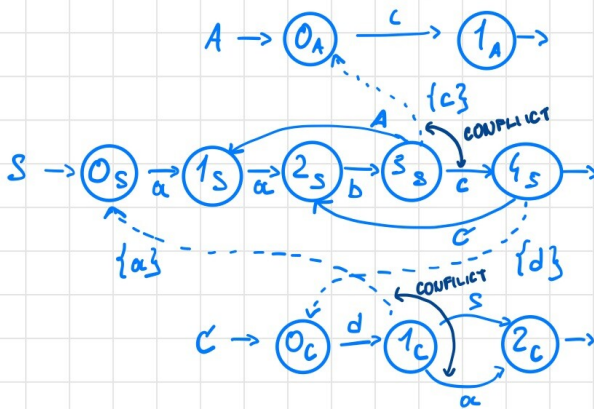
look ahead contains all
terminals that can occur after
 M_1 terminates





Is it ELR(1) condition verified? Yes, no conflicts can be found
 Is it ELL(1) condition verified?

- if we have the pilot: 1. check that there is no leftmost recursion $\rightarrow O \xrightarrow{A} O$
- 2. verify that STP is verified
- 3. verify ELR(1) condition
- if we do not have the pilot: 1. build the power control graph
- 2. verify that the net remains unambiguous with call-arcs added



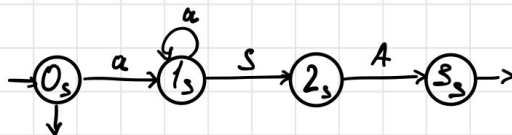
The grammar is not ELL(1) because it doesn't respect STP.

17/11/22

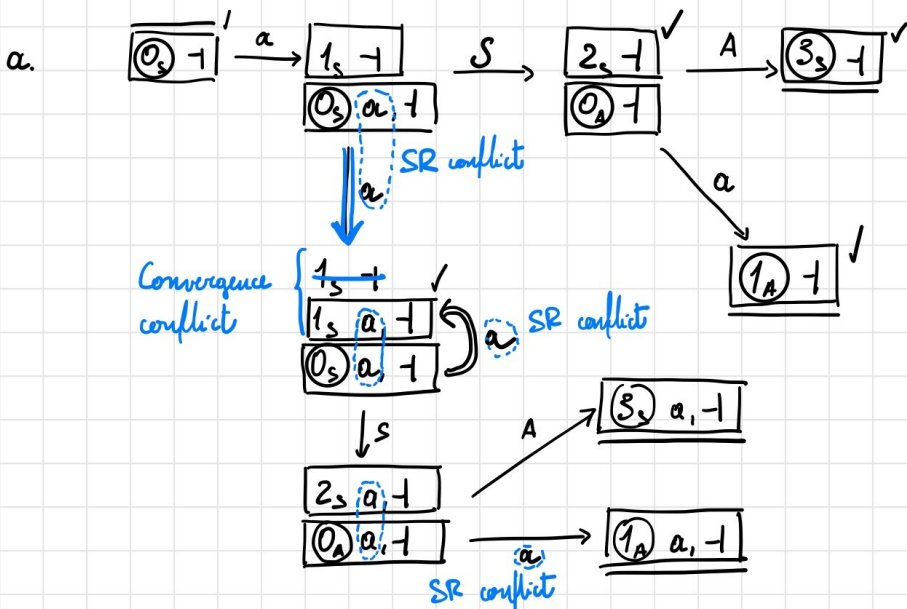
14 JAN 2020

$\Sigma = \{a\}$
 $V = \{A, S\}$

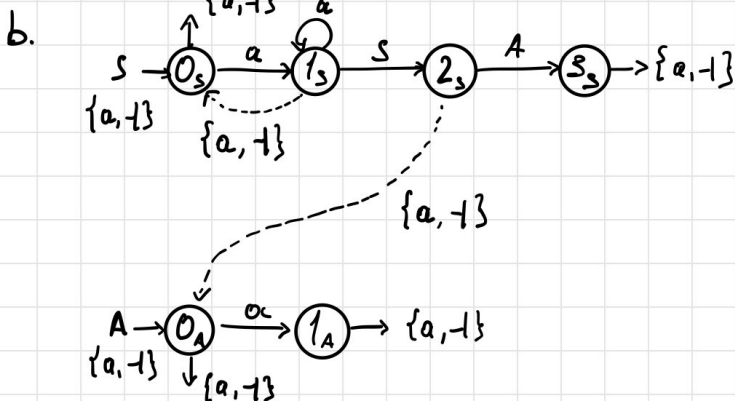
$S \rightarrow \epsilon / a^+ SA$



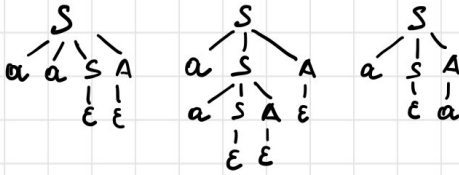
$A \rightarrow \epsilon / a$



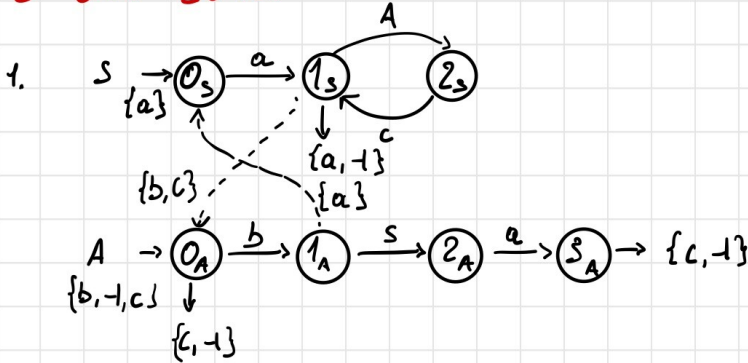
The grammar is not ELR(1) and neither ELL(1)



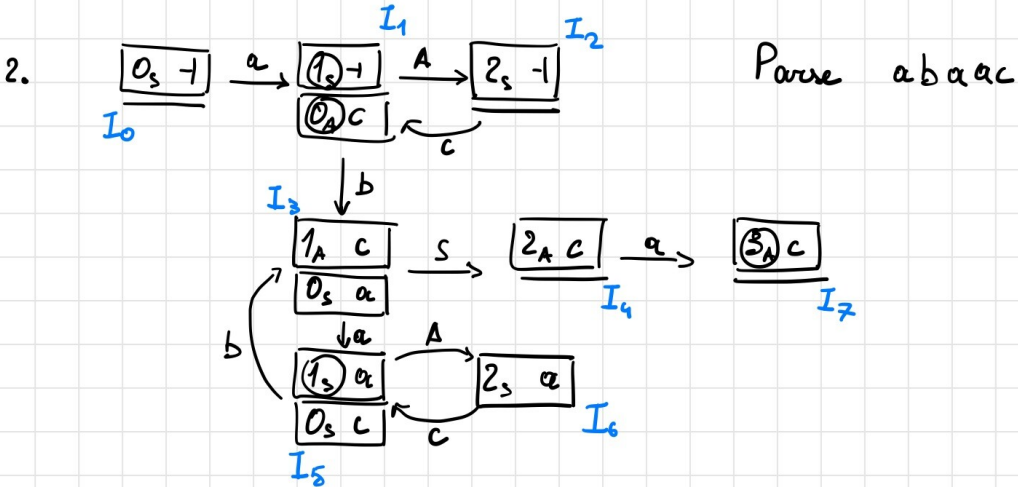
C. "aa"



2 OCT 2012



Grammar is ELL(1).



Stack

I_0

$I_0 a I_1$

$I_0 a I_1 b I_3$

$I_0 a I_1 b I_3 a I_5$

$I_0 a I_1 b I_3$

$I_0 a I_1 b I_3 S I_4$

$I_0 a I_1 b I_3 S I_4 a I_7$

$I_0 a I_1$

$I_0 a I_1 A I_2$

$I_0 a I_1 A I_2 c I_1$

I_0

ACCEPT

String left

abac-

baac-

aac-

ac-

(S) ac-

ac-

c-

(A) c-

c-

-

(S) -

Action

shift a

shift b

shift a

reduce $a \rightarrow S$

shift S

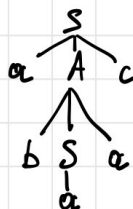
shift a

reduce $bSA \rightarrow A$

shift A

reduce c

reduce $aAc \rightarrow S$



3. Build the recursive-descent parser.

prog Parser

cc = next

call S

if $cc \in \{-1\}$ then accept

else error

end program

procedure S

0s: if $cc \in \{-1\}$ then cc = next; goto 1s

error

1s: if $cc \in \{b, c\}$ then call A; goto 2s

if $cc \in \{a, -1\}$ then return

error

2s: •

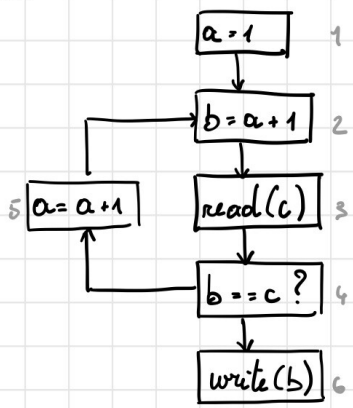
•

•

15/12/22

13-07-2016 4.2

a.



$$in(p) = uses(p) \cup (out(p) \setminus def(p))$$

$$out(p) = \bigcup_{q \in succ(p)} in(q)$$

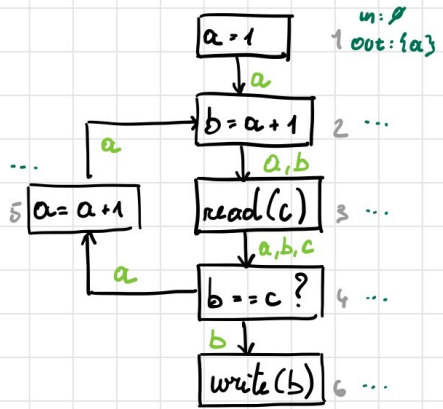
#	def	uses	IN	OUT	OUT	IN	OUT	IN	OUT
1	a	-	$out(1) \setminus \{a\}$	$in(2)$	-	-	a	-	a
2	b	a	$\{a\} \cup (out(2) \setminus \{b\})$	$in(3)$	-	a	-	a	b
3	c	-	$out(3) \setminus \{c\}$	$in(4)$	-	-	b, c	b	a, b, c
4	-	b, c	$\{b, c\} \cup (out(4) \setminus \emptyset)$	$in(5) \cup in(6)$	-	b, c	a, b	a, b, c	a, b
5	a	a	$\{a\} \cup (out(5) \setminus \{a\})$	$in(2)$	-	a	a	-	a
6	-	b	$\{b\} \cup (out(6) \setminus \emptyset)$	$\emptyset$	-	b	-	b	-

INITIALIZED AS $\emptyset$

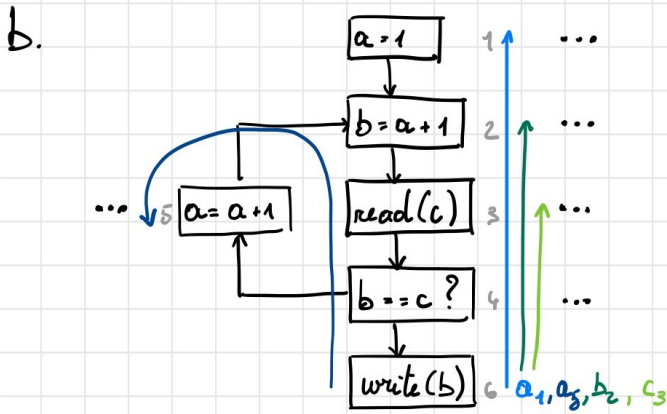
1st iter

IN	OUT	IN	OUT
-	a	-	
a	a, b	a	
a, b	a, b, c	a, b	
a, b, c	a, b	a, b, c	
a	a	a	
b	-	b	

equal $\Rightarrow$ finish iteration



on, on ou equivalent annotations



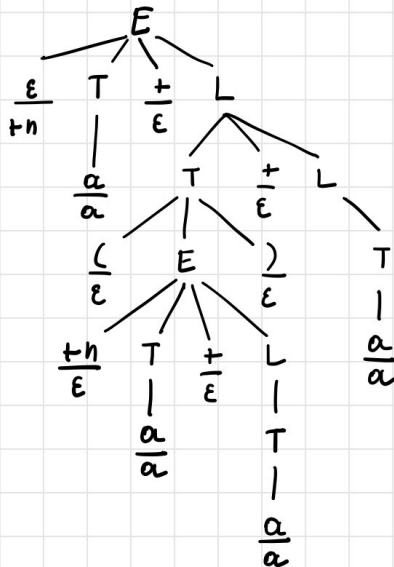
2019 - 01 - 13

4.2

a.

$$G_s \begin{cases} E \rightarrow T+L \\ L \rightarrow T+L \\ L \rightarrow T \\ T \rightarrow (E) \\ T \rightarrow a \end{cases} \Rightarrow G_r \begin{cases} E \rightarrow +n T L \\ L \rightarrow T L \\ L \rightarrow T \\ T \rightarrow E \\ T \rightarrow a \end{cases}$$

$a+(a+a)+a \rightarrow$ converted into expression of type $+naa$
 $\rightarrow +na+naaa$



b. Compute τ' that chooses n if there are only 2 operands (changing G_s is allowed).

$$E \rightarrow T + T$$

$$E \rightarrow T + T + L$$

$$L \rightarrow T + L$$

$$L \rightarrow T$$

$$T \rightarrow (E)$$

$$T \rightarrow a$$

$$E \rightarrow + T T$$

$$E \rightarrow + n T T L$$

$$L \rightarrow T L$$

$$L \rightarrow T$$

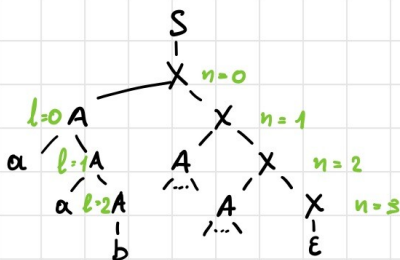
$$T \rightarrow E$$

$$T \rightarrow a$$

2018-02-7 4.2

1. $S \rightarrow X$
2. $X \rightarrow A X$
3. $X \rightarrow \epsilon$
4. $A \rightarrow a A$
5. $A \rightarrow b$

name	type	domain	symbol	
l	left	int	A	len of list
n	right	int	X	req len of A
e	left	bool	X, S	validity



#		
1	$S_0 \rightarrow X_1$	$n_1 \leftarrow 0; e_0 \leftarrow e_1$
2	$X_0 \rightarrow A_1 X_2$	$n_2 \leftarrow n_0 + 1; e_0 \leftarrow (l_2 == n_0) \wedge e_2$
3	$X_0 \rightarrow \epsilon$	$e_0 \leftarrow t$
4	$A_0 \rightarrow a A_2$	$l_0 \leftarrow l_1 + 1$
5	$A_0 \rightarrow b$	$l_0 \leftarrow 0$